Understanding Language with Machines
Senior Comprehensive 2025

Julia Lizak
Advisor: Dr. Kuter

December 9, 2025

Contents

1 Introduction **1**

1.1 Motivation 1

1.2 Project Scope and Evolution 1

1.3 Goals of the Project 1

2 Background **3**

2.1 Logistic Regression 3

2.2 Odds and Log-Odds 4

2.3 Logistic Model and Coefficient Interpretation 6

2.4 Text Representation 7

2.5 Vectorization Approaches 9

2.6 Inverse Document Frequency (IDF) 12

2.7 Combine TF and IDF 15

3 Methods **16**

3.1 Data Collection and Preprocessing 16

3.2 Model specification 17

3.3 Implementation 21

4 Results **22**

4.1 Vocab and Feature Analysis 22

4.2 Classification Performance 23

4.3 Interpretation 24

5 Discussion **25**

5.1 Strengths 25

5.2 Limitations 25

6 Conclusion **26**

Appendices **28**

A Conceptual Walk-Through: From Raw Text to Prediction **28**

A.1 Start with Raw Text 28

A.2 Term Frequency (TF) 29

A.3 Inverse Document Frequency (IDF) 30

A.4 Combining into TF-IDF Weights 30

A.5 Logistic Regression Model Setup 31

A.6 Interpreting Coefficients through Odds Ratio 32

B Implemented Model Using Python **32**

C Setup and Cleaning Code **33**

D Model Training Code (Logistic Regression) **34**

References **37**

1 Introduction

1.1 Motivation

Understanding how computers process language in a mathematical way has become central to data science and modern artificial intelligence. Language surrounds nearly every aspect of our lives, especially on campus, from course catalogs and advising notes, to policy documents and student communications. Text is structured and interpretable for humans, but not machines. Turning the text into machine-interpretable information enables tools such as search systems, academic recommendations, and data–driven analytics.

This project examines how foundational statistical and computational methods can represent text numerically and use those representations to make meaningful predictions. Rather than focusing directly on advanced neural architectures, the goal is to understand the mathematical principles that underlie more complex language models.

1.2 Project Scope and Evolution

This Senior Comprehensive project originally focused on exploring a broad range of models used for language understanding, including Recurrent Neural Networks (RNNs), Transformers, and Large Language Models (LLMs). However, after deeper discussions and faculty feedback, we realized the importance of building a strong mathematical foundation before moving on to more complex systems. For this reason, the scope of this project shifted towards a focused and interpretable model: *binary text classification using logistic regression.* This change allowed us to analyze and learn the essential relationship between probability, linear modeling, and the way text is represented numerically.

1.3 Goals of the Project

For this project, a dataset consisting of course descriptions was collected from the Saint Mary's College (SMC) Bulletin. Each course was labeled as either **MATH** (1) or **ENLT** (0), forming

a balanced set of short, paragraph-length documents. The objective is to train a classifier that can correctly predict whether a listed SMC course description belongs to Mathematics or English Literature based solely on its text description.

The project consists of four main goals. First, it is important to understand *logistic regression* both conceptually and mathematically, starting from linear regression and building toward the logit transformation that maps labels of 0 and 1 to their probabilities between 0 and 1. Following that is applying the *TF-IDF (Term Frequency–Inverse Document Frequency)* method to convert course descriptions into numerical feature vectors that provide the importance of each term. Next is implementing and evaluating a simple, interpretable *binary classifier* trained on these TF-IDF features, using logistic regression. Finally, the last goal is to analyze model performance, interpret the coefficients, and connect this foundation to larger concepts used in modern natural language processing (e.g., embeddings, RNNs, and Transformers).

In this paper, we first introduce the mathematical background of logistic regression, beginning with linear regression and progressing through the log-odds and logistic function that form the model's probabilistic foundation. We then consider how text data is preprocessed, tokenized, and vectorized. Next, we move on to TF-IDF weighting scheme and its motivation. We will implement a logistic regression model on the course description dataset, including results, evaluation, and interpretation. We then conclude with a reflection on the change of scope and the connection between these mathematical foundations and large-scale language models.

Throughout this paper, we use the following notation:

P	Probability of an event
p	number of predictors
N	sample size
X	Vector of input features (predictors)
x_i	The ith feature (i.e., TF–IDF weight of a specific term)
Y	Binary output variable (e.g. 1 = MATH, 0 = ENLT)
$p(X)$	short-hand for $p(X) = P(Y = 1\|X)$
β_0	Intercept term of logistic regression model
β_i	Coefficient representing the effect of feature x_i
logit	Log odds transformation
$tf_{t,d}$	Term Frequency

2 Background

2.1 Logistic Regression

In a binary classification setting, there is an interest in the probability that an observation belongs to a specific class. We are interested in whether a course description should be classified as *Math* or *English*. We consider the probability that a course description belongs to the *Math* category, probability $P(Y = 1)$. Specifically, we focus on the conditional that is,

$$p(X) = P(Y = 1 \mid X)$$

where X is the vector of features derived from the text. The central objective is therefore to model how these predictors relate to the probability, while ensuring that the output remains within the valid range: $0 < p < 1$.

Linear regression models a continuous outcome variable as a weighted sum of predictors. A natural starting point is the linear model

$$Y = \beta_0 + \beta_1 X$$

When using the linear model, the objective is to predict a continuous outcome variable, such as a student's GPA or a housing price, using a weighted combination of input features. However,

applying this model directly to a binary response is problematic. The expression $\beta_0 + \beta_1 X$ may take any value in $(-\infty, \infty)$, while the quantity we wish to model, the label Y or its probability of equaling 1, must be between 0 and 1. As a result, linear regression does not provide valid estimates in the context of binary outcomes. Focusing on the goal to model the probability $p(X)$, we require a transformation that maps this probability to a scale that can take any real value so that we may utilize the linear model approach. This is done by considering the *odds* of the event $Y = 1$, which provide a more suitable intermediate quantity for constructing a linear model.

2.2 Odds and Log-Odds

The first step is to express probability in terms of *odds*. Odds are defined as the ratio of the probability of an event to its complement. In our context, we consider $Y = 1$ a "success" because we are focused on predicting when a course description is MATH. The odds of a success are given by:

$$\text{odds} = \frac{P(Y=1)}{1 - P(Y=1)} = \frac{\text{probability of success}}{\text{probability of failure}}.$$

For example, if we were to have a probability of success equal to 0.7, the odds of success are:

$$\text{odds} = \frac{0.7}{0.3} \approx 2.3.$$

This means that success is over twice as likely as failure.

If we were to look at another example, say the probability of success = 0.3, we would get

$$\text{odds} = \frac{0.3}{0.7} \approx 0.43.$$

In this case, a success is less likely.

To fully interpret how much the probability of success is less than probability of failure, consider the following:

$$\frac{P(failure) - P(success)}{P(failure)}$$
$$= 1 - \frac{P(success)}{P(failure)} = 1 - odds$$

$$= 1 - 0.43$$

$$= 0.57$$

Now, we can see that there is a 57% decrease from probability of failure to the probability of success. In other words, the probability of success is 57% less likely than the probability of failure.

0.3 is 43% of 0.7

OR 57% less.

In general, odds are interpreted as follows:

odds > 1: (odds - 1) ×100 gives percentage increase from the probability of failure to success

odds = 1: the probability of failure equals the probability of success

odds < 1: (1 - odds) ×100 gives the percentage decrease from the probability of failure to success

While odds are useful, they still only range from 0 to ∞. This does not resolve the core issue that prevented linear regression from being appropriate in the first place: a linear model requires a response that can take any value $(-\infty, \infty)$. Even though odds remove the restriction of staying within $(0, 1)$, they remain positive and therefore cannot be related to the predictors through an unrestricted linear expression, like $\beta_0 + \beta_1 X$. To get a quantity that is compatible with a linear model, we apply the natural logarithm to the odds. This produces the *log-odds* (or *logit*) given by

$$\text{logit}(P(Y = 1)) = \ln\left(\frac{P(Y = 1)}{1 - P(Y = 1)}\right).$$

The logit transformation maps probabilities in $(0, 1)$ to the entire real line $(-\infty, \infty)$, allowing us to use a linear relationship between the predictors and the transformed probability. This transformation is fundamental to logistic regression because it linearizes the multiplicative nature of odds and allows the coefficients of the model to be interpreted additively on the log-odds scale.

2.3 Logistic Model and Coefficient Interpretation

We now use the logit transformation to construct a linear model for the transformed probability. This models results from performing linear regression on the logit transformation of the probability $p(X)$. Using the logit transformation, we can now relate the transformed probability to the predictors through a linear model. In the general case with predictors $x_1, \ldots, x_p$, we model

$$\text{logit}(p(X)) = \ln\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_p. \tag{1}$$

This linear model on the log-odds scale is the foundation of logistic regression.

Solving for $p(X)$ in Equation (1) yields the *logistic function*:

$$p(X) = \frac{e^{\beta_0+\beta_1 x_1+\cdots+\beta_n x_p}}{1+e^{\beta_0+\beta_1 x_1+\cdots+\beta_n x_p}}. \tag{2}$$

Equation (1) and (2) are equivalent. The logistic function is obtained by algebraically inverting the linear *logit* model to express the probability.

The coefficients $beta_i$ determine how each predictor x_i influences the log-odds of the outcome. Specifically, β_i represents the change in log-odds associated with a one-unit increase in x_i, keeping all other predictors constant. If $\beta_i > 0$, increasing x_i increases the log-odds of $Y = 1$, and therefore increases $p(X)$. If $\beta_i < 0$, increasing x_i decreases the log-odds and thus decreases $p(X)$.

Because the log-odds scale is logarithmic, it is often more interpretable to exponentiate the coefficients. The quantity e^{β_i} is the *odds ratio* associated with a one-unit increase in x_i:

$$e^{\beta_i} = \frac{\text{odds after a one-unit increase in } x_i}{\text{original odds}}.$$

For example, if $\beta_i = 0.7$, then $e^{0.7} \approx 2.0$, meaning that a one-unit increase in x_i doubles the odds of the course being classified as Math.

Interpreting logistic regression through both the log-odds and the odds ratio provides an understanding of how each term contributes to the model. Since logistic regression outputs probabilities, the model estimate for $p(X)$ can be interpreted as the estimated probability that a course belongs

to the Math category. Once we have these probabilities, there is a "cutoff" (a standard threshold of $p = 0.5$ is applied) that is used to convert these probabilities into binary decisions, which we will discuss later.

2.4 Text Representation

Before applying logistic regression to course descriptions, the raw text descriptions must be converted into numerical features that a machine learning model can use. This process begins with basic preprocessing steps that standardize the text and reduce noise. The cleaned text is then tokenized into individual words, or *tokens*, which are the building blocks for building these numerical representations. Preprocessing is what helps to make a consistent format that can be easily analyzed. Each description is cleaned by converting the text to lowercase and removing punctuation. In our case, all course descriptions collected should be lower-cased to ensure that words such as "Calculus" and "calculus" will be treated the same way. Along with capitalization, marks like punctuation, numbers, and extra spaces may also be removed, depending on the context of the model. For this project, preprocessing consisted of lowercasing the text and removing punctuation to ensure that the descriptions followed a uniform format.

Following preprocessing is *tokenization*. The cleaned text is then split into individual words, or *tokens*, which serve as the building blocks for constructing numerical representations. For example, consider the course title "Introduction to Statistics".
The phrase after the preprocessing step looks like this:

"introduction to statistics"

and it becomes the list

[`introduction`, `to`, `statistics`].

Each one of the tokens is then treated as a feature candidate that can contribute to the overall representation of the document, or in our case, the description.

Once tokenization is complete, any further preprocessing decisions will depend on the purpose of the model. For example, some workflows remove very common words, which are known as *stop words*, such as "the," "and," or "of." Since these words tend to appear frequently across all documents and typically do not help us when distinguishing between categories, it is reasonably valid to remove them. On the other hand, words that are more unique to a certain category (i.e. "Calculus" for MATH or "Poetry" for ENLT) will then begin to have a greater importance per category.

Example 2.1. Let's use Math 103 as an example to demonstrate how preprocessing works. The course description is:

> This course studies polynomial, rational, exponential, logarithmic, and trigonometric functions from the symbolic, numeric, and graphical perspectives. The emphasis on these concepts will provide solid preparation for a college-level calculus course. This does not fulfill the Mathematical Arts requirement of the Sophia Program. Prerequisite: MATH 101 or recommendation of Math Placement Advisor.

Removing the punctuation and making the words lowercase returns:

> this course studies polynomial rational exponential logarithmic and trigonometric functions from the symbolic numeric and graphical perspectives the emphasis on these concepts will provide solid preparation for a college-level calculus course this does not fulfill the mathematical arts requirement of the sophia program prerequisite math 101 or recommendation of math placement advisor

The tokens will then be:

```
[this, course, studies, polynomial, rational, exponential, logarithmic, and,
      trigonometric, functions, ..., college-level, calculus, course]
```

After removing stop words and keeping hyphenated words, it returns:

```
[studies, polynomial, rational, exponential, logarithmic, trigonometric,
                functions, college-level, calculus, ...]
```

The same process would be implemented for any other Math or English description. These cleaned token lists are now the inputs for the vectorizer.

2.5 Vectorization Approaches

After preprocessing and tokenization, each course description now exists as a sequence of tokens. As we know, logistic regression cannot process the raw text inputs, it needs numerical input. The next step is *vectorization*, where we transform the tokens into a structured numerical format.

In the simplest form, *vectorization* counts how many times each words appears in a description and arranges the counts into a vector. Each entry in the vector corresponds to a word from the entire vocabulary of the *corpus* (or body), the collection of all course descriptions. When cycling through all descriptions, if a word appears in a document, the count for that specific word increases; if it does not appear, the corresponding count remains zero. Applying this process to every description produces the document–term matrix in which each row corresponds to a document and each column to a word. Because most descriptions use only a small subset of the vocabulary, this results in a complex but highly sparse matrix.

We implement a simple and widely used numerical representation of text, the *bag-of-words* model. In this model, each document is treated as a collection of words without considering word order, grammar, or sentence structure. The document is represented with a vector where each entry is the number of times each word appears. For example, common words such as "course," "students," or "write" may appear frequently in both Math and English descriptions but do not help distinguish between the two categories. While straightforward, raw word counts can be overpowered by very common words and may not accurately reflect how informative a term is for classification.

To account for how informative a word is, not only by its frequency within a document but also by how rare it is across documents—the idea of *weights* becomes important. This leads to the Term Frequency–Inverse Document Frequency (TF–IDF) representation, which scales each word by both how often it appears in a description and how distinctive it is across the corpus. TF–IDF helps emphasize category-specific words while reducing the influence of generic terms that occur in many documents.

Example 2.2. The following example illustrates how counts and sparsity appear in a bag-of-words representation, using two actual course descriptions from the **smc_courses** dataset.
Consider the following descriptions:

> d_1 (MATH 103): This course studies polynomial, rational, exponential, logarithmic, and trigonometric functions from the symbolic, numeric, and graphical perspectives. The emphasis on these concepts will provide solid preparation for a college-level calculus course. This does not fulfill the Mathematical Arts requirement of the Sophia Program. Prerequisite: MATH 101 or recommendation of Math Placement Advisor.
>
> d_2 (ENLT 216): What role do literary works play in advocating for a more just society? Can literature provide a platform for social exploration? An opportunity to engage the emotions of the public? A space for disruption and critique? What are the limitations of literary representations of social justice issues? This course uses these guiding questions to examine works of poetry, fiction, and nonfiction.

To build a bag-of-words representation, we first choose a small vocabulary of distinctive terms that actually appear in these descriptions

$$V = [\texttt{calculus}, \texttt{polynomial}, \texttt{trigonometric}, \texttt{literature}, \texttt{justice}, \texttt{poetry}].$$

For each document d_i, the bag-of-words vector records how many times each term in V appears in that description. For d_1 and d_2 this gives

	`calculus`	`polynomial`	`trigonometric`	`literature`	`justice`	`poetry`
d_1 (MATH 103)	1	1	1	0	0	0
d_2 (ENLT 216)	0	0	0	1	1	1

In vector form, we can write

$$\text{bag-of-words}(d_1) = [1, 1, 1, 0, 0, 0], \qquad \text{bag-of-words}(d_2) = [0, 0, 0, 1, 1, 1].$$

Most words appear in only one description, which gives very few non-zero entries. This tells us why the resulting matrix is sparse and why TF–IDF is useful. It assigns higher weights to distinctive terms that appear in only a small number of descriptions, while down-weighting words that are frequent but uninformative.

Term Frequency (TF). The counting procedure demonstrated above is the basis of *term frequency*, which measures how often a word appears within a single document. For a word t and a document d, the term frequency is denoted as

$$tf_{t,d}.$$

The simplest approach is to assign the weight to be equal to the number of occurrences of token t in the document d, as follows:

$$tf_{t,d} = \text{number of times token } t \text{ appears in document } d.$$

In some cases, the count of words is normalized by the total number of words in the document, to account for differences in document length of the document:

$$tf_{t,d} = \frac{\text{number of times } t \text{ appears in } d}{\text{total number of terms in } d}.$$

The *term frequency* gives the importance of a word within a single course description. A higher TF indicates that the word is more prominent within the document. Thus, it provides the basic frequency information used in the bag-of-words model, and will serve as the first component of TF–IDF.

Example 2.3. Let's walk through the TF steps on a few dataset terms, using the same course description from *Example 2.2*: ENLT 216. The token `literature` appears just once, while the token `works` appears twice. Their TF values are

$$tf_{\texttt{literature},\, d_2} = \frac{1}{34} \approx 0.029, \qquad tf_{\texttt{works},\, d_2} = \frac{2}{34} \approx 0.059.$$

These TF values show how term frequency reflects the prominence of a token within a single document. Tokens that occur more frequently receive larger TF weights, while less frequent tokens receive smaller weights.

2.6 Inverse Document Frequency (IDF)

The next component of TF-IDF is *Inverse Document Frequency*, which provides a measure of how common or rare a word is across the entire collection of documents. If a term were to appear in many documents, it becomes less and less useful for distinguishing between the two categories. We let N be the total number of documents, and df_t be the number of documents containing term t. Then the *inverse document frequency* is defined as:

$$idf_t = \log\left(\frac{N}{df_t}\right).$$

Using TF-IDF weights as input data The TF-IDF weights act as the numeric predictors. Each token's weight in a document is computed as

$$x_i = tf_{t,d} \times idf_t.$$

The more documents a token appears in, the smaller its IDF value becomes. One way to see why the idf_t formula has this shape is to express the proportion of documents containing the token as a probability. We can estimate U_t as

$$U_t = \frac{df_t}{N}, \qquad 1 - U_t = \frac{N - df_t}{N}.$$

U_t is the probability that a randomly selected document contains the token, and $1 - U_t$ is the probability it does not. The ratio

$$\frac{1 - U_t}{U_t} = \frac{N - df_t}{df_t}.$$

represents the odds of a document not containing the token compared to containing it. Taking the logarithm of this odds ratio gives

$$\log\left(\frac{1 - U_t}{U_t}\right) = \log\left(\frac{N}{df_t}\right)$$

which matches the IDF expression. Thus, the IDF value can be viewed as the log-odds of a term being absent rather than present across the corpus. Rare terms therefore receive larger IDF weights, while common terms receive weights close to zero. And so, we can summarize this relationship as:

$$\text{weights: } \log\left(\frac{1-U_t}{U_t}\right) \approx \log\left(\frac{N}{df_t}\right) \leftarrow (idf_t)$$

This shows how IDF behaves like the log-odds of a document *not* containing the term versus containing it.

The logarithm in the IDF formula serves two important purposes that make the weighting more stable and interpretable.

First, it balances the influence of rare words, so that they do not hold too much power in the representation. Without the log, a term that appears in only one document would give a ratio such as $\frac{N}{1}$, which could be a very large IDF value. Applying the logarithm compresses these large values, preventing rare terms from becoming too overpowering.

Second, the logarithm converts multiplicative differences in document frequency into additive differences in log space. This helps to make the weights more interpretable. For example, suppose there are $N = 100$ documents in the corpus, and consider the two terms t_1 and t_2 with different document frequencies. If t_1 appears in 10 of the documents, then $df_{t_1} = 10$, and

$$\frac{N}{df_{t_1}} = \frac{100}{10} = 10 \quad \Rightarrow \quad idf_{t_1} = \log(10) = 1.$$

If t_2 appears in all 100 documents, then $df_{t_2} = 100$, and

$$\frac{N}{df_{t_2}} = \frac{100}{100} = 1 \quad \Rightarrow \quad idf_{t_2} = \log(1) = 0.$$

The ratio $100/10 = 10$ suggests that one term is ten times more common than the other. The logarithm expresses this tenfold difference as a difference of only one unit in the IDF scores (from 1 to 0), which is far more interpretable in weighting schemes. This smaller difference is easier for the model to interpret and work with because logistic regression operates on a linear scale. Additive

changes of about one unit integrate easier into the model, whereas a "ten times" difference would create large and uneven weight contributions.

Thus, the IDF value decreases weight for words that appear in many documents and increases for those that appear in only a few. This overall helps the model to focus on words that are more unique or distinctive per category.

Example 2.4. To see how IDF distinguishes informative words from common ones in our dataset, consider the three tokens: `calculus`, `geometry`, and `course`. The `smc_courses` dataset contains $N = 20$ documents $(d_1, \ldots, d_{20})$

The token `geometry` appears in only one description.

$$df_{\texttt{geometry}} = 1, \quad idf = \log\left(\frac{20}{1}\right) \approx 2.996.$$

The token `calculus` appears in two descriptions.

$$df_{\texttt{calculus}} = 2, \quad idf = \log\left(\frac{20}{2}\right) \approx 2.303.$$

The token `course` appears in eleven descriptions.

$$df_{\texttt{course}} = 11, \quad idf = \log\left(\frac{20}{11}\right) \approx 0.598.$$

These values show how IDF behaves and the effect of IDF on real tokens from the corpus. A distinctive term that appears in only a few descriptions (such as `geometry` receives the largest IDF weight, tokens that appear in a small number of descriptions (such as `calculus` receives a moderate weight, and common tokens that appear throughout the dataset (such as `course` receive a much smaller weight. In this way, IDF emphasizes terms that are unique or distinctive, while down-weighting terms that are common across the entire corpus and provide little information for distinguishing MATH from ENLT descriptions.

2.7 Combine TF and IDF

To obtain the TF-IDF weight of a term t in a document d, we combine the information provided by term frequency (TF) and inverse document frequency (IDF). TF reflects how important a term is within a single document, while IDF reflects how distinctive that term is across the entire corpus. The TF-IDF weight is defined as

$$\text{tf-idf}_{t,d} = tf_{t,d} \times idf_t.$$

This weighting scheme comes from information retrieval, where the objective is to measure the relevance of a term based on both its presence in a document and its distribution across the entire collection. In information retrieval, documents are compared based on how informative their terms are, and TF–IDF provides a principled way to quantify that informativeness. The TF-IDF process fits naturally into the earlier steps of text preprocessing (tokenizing, vectorizing, etc.), since it transforms raw term counts into meaningful numerical weights. The resulting value represents the weight of term t in the document d. A term receives a high weight when it appears many times in a document but in only a few documents across the corpus, a lower weight when it appears infrequently in that document or many documents overall, and the lowest weight when it appears in almost all documents, meaning it carries little value for distinguishing between categories.

Example 2.5. To illustrate how TF and IDF combine, consider the tokens from the datatset. In the MATH 103 description, the token `calculus` appears once in the 34 token document, which gives

$$tf_{\texttt{calculus},d_1} = \frac{1}{34} \approx 0.029.$$

Using the corpus of $N = 20$ descriptions, its document frequency is $df_{\texttt{calculus}} = 2$, so

$$idf_{\texttt{calculus}} = \log\left(\frac{20}{2}\right) \approx 2.303.$$

Thus, the TF-IDF weight is

$$\text{tf-idf}_{\texttt{calculus},d_1} = 0.029 \times 2.303 \approx 0.067.$$

In comparison, the token `course` appears once in the same document, but 11 total descriptions. The IDF is much smaller:

$$idf_{\texttt{course}} = \log\left(\frac{20}{11}\right) \approx 0.598,$$

$$\text{tf-idf}_{\texttt{course},d_1} = 0.029 \times 0.598 \approx 0.017.$$

Even though both tokens occur once in the MATH 103 description, `calculus` receives a larger TF-IDF weight because it appears in less descriptions across the corpus than course.

3 Methods

3.1 Data Collection and Preprocessing

This project uses the `smc_courses.csv` dataset, which contains twenty course descriptions: ten from Mathematics(MATH) and ten from English Literature (ENLT). Figure 3.1 shows a small preview of the dataset.

Figure 3.1: Preview of the smc-courses dataset

	A	B	C	D	E	F	G	H	I	J	K	L
1	label	course_id	title	description								
2	ENLT	ENLT 106W	Language and	A range of courses taught in tandem with "W" courses in other disciplines. Students may earn three literature hours ar								
3	ENLT	ENLT 112	FYS: Mystery	Mystery is one of the most enduring features of fiction and nonfiction alike. In this literature class, we will consider not								
4	ENLT	ENLT 151	Intoduction t	Introduction to Literature (3/4) This course introduces students to the skills of reading and writing that enable us to ap								
5	ENLT	ENLT 211	Animals in Lit	This interdisciplinary course explores how literature shapes and reflects human responses to nonhuman animals, usir								
6	ENLT	ENLT 213	Environment	This course studies writings that engage their readers with the natural environment. We will learn the ways of reading								
7	ENLT	ENLT 216	Literature of	What role do literary works play in advocating for a more just society? Can literature provide a platform for social expl								
8	ENLT	ENLT 217	Contempora	Readings of contemporary women writers since the 1960s.								
9	ENLT	ENLT 220	Literature an	What can literature tell us about experiences of health, illness, and health care? How do literary works represent and								
10	ENLT	ENLT 224	Sorcery and I	From Homer and Dante to Anthony Burgess and Anne Rice, this course examines one of the oldest and most fascinatir								
11	ENLT	ENLT 232	Arthurian Lit	An exploration of the myth of Arthur from its medieval roots to the present day. While this course will focus on literary								
12	MATH	MATH 100	Problem-Solv	Intensive study of the problem-solving process. Algebraic, patterning, modeling, and geometric strategies are explore								
13	MATH	MATH 101	College Alget	This course will cover topics in algebra that are needed for future courses. Topics will include basic algebraic concept								
14	MATH	MATH 102	Liberal Arts M	Mathematical modeling through the use of graph theory. Topics include graphs, directed graphs, trees, matchings and								
15	MATH	MATH 103	Precalculus	This course studies polynomial, rational, exponential, logarithmic, and trigonometric functions from the symbolic, nur								
16	MATH	MATH 104	Finite Mathe	Set theory, counting techniques, probability, random variables, expected value, variance, standard deviation, and lin								
17	MATH	MATH 107	Mathematics	This course develops and applies mathematical concepts and tools to quantitatively explore sustainability issues. Top								
18	MATH	MATH 108	Matrices and									
19	MATH	MATH 110	Modern Geor	Finite geometries. Transformational geometry with an introduction to fractals. Euclidean geometry, including classica								
20	MATH	MATH 113	Survey of Cal	A one semester introduction to differential and integral calculus designed primarily for liberal arts students and those								
21	MATH	MATH 114	Introduction	Introduction to basic sampling and experimental design. Basics of probability, random variables, and probability distr								

Before modeling, the text was cleaned by converting all entries to strings, removing missing values, and trimming extra whitespace. A TF-IDF vectorizer with English stop-word removal was then applied to learn the vocabulary from the twenty descriptions and to compute the TF-IDF weights for each term. This produced a sparse document-term matrix where each row corresponds to a course description and each column a word in the vocabulary.

After computing TF-IDF values for all terms, each document can now be represented as a vector of these weights. We denote the TF-IDF vector for document d by

$$\text{vec}(d) = [w_{1,d}, w_{2,d}, \ldots, w_{|V|,d}],$$

where $|V|$ is the size of the vocabulary and $w_{i,d}$ is the TF-IDF weight of term i in document d.

Because each course description uses only a small subset of the vocabulary, most values in $\text{vec}(d)$ are zero. The representation of a set of documents as vectors in a common vector space is known as *vector space model*.

3.2 Model specification

Before fitting the model, it is useful to see how TF-IDF already highlights differences between Math and English Literature descriptions in this dataset. For example, the term `geometry` appears in only a few Mathematics course descriptions and rarely (if ever) in English Literature descriptions. This gives the term a reasonably high term frequency (TF) within certain MATH documents, and because it appears in only a small number of documents, it has a low document frequency. Together, these produce a relatively large TF-IDF weight for `geometry`, which makes it a useful indicator of the Math category. On the other hand, the word `course` appears in just about every course description, both MATH and ENLT. Although its term frequency may be high in many documents, its document frequency is also higher. This gives it an IDF value most likely close to zero, thus the TF-IDF weight is small. These examples illustrate how TF-IDF emphasizes distinctive tokens while down-weighting generic ones, thus forming the foundation for the logistic regression model.

Once each description is represented by a TF-IDF vector, these weights become the numerical features input to the logistic regression model. Each coefficient β_i in the logistic regression equation corresponds to a specific term in the vocabulary, meaning the model learns how each word contributes to distinguishing between the two categories.

Consider two descriptions from the dataset:

> MATH 103: This course studies polynomial, rational, exponential, logarithmic, and trigonometric functions from the symbolic, numeric, and graphical perspectives. The emphasis on these concepts will provide solid preparation for a college-level calculus course. This does not fulfill the Mathematical Arts requirement of the Sophia Program. Prerequisite: MATH 101 or recommendation of Math Placement Advisor.

In this description, the words `emphasis` and `prerequisite` appear with TF-IDF weights determined by the vectorizer. The logistic regression model assigns these words positive coefficients (stated in the *Results* section) in the fitted model:

$$\beta_{\text{emphasis}} = 0.218, \qquad \beta_{\text{prerequisite}} = 0.210.$$

Since both coefficients are positive, increases in the TF-IDf weights will increase the log-odds that the description belongs to the MATH category.

> ENLT 216: What role do literary works play in advocating for a more just society? Can **literature** provide a platform for social exploration? An opportunity to engage the emotions of the public? A space for disruption and critique? What are the limitations of literary representations of social justice issues? This course uses these guiding questions to examine works of poetry, fiction, and nonfiction.

In contrast, the token `literature` appears with a strongly negative coefficient in the fitted model:

$$\beta_{\text{literature}} = -0.325.$$

The negative coefficient decreases the log-odds of predicting MATH, pushing the model toward an ENLT classification when this term appears with a nonzero TF-IDF weight. Overall, the model learns a weight combination of TF-IDF features that best separate the two categories in the dataset.

Example 3.1. A quick representation of combining the terms, stemming from the previous examples. These values in this table help to show how distinctive terms get larger weights, even with a medium frequency inside the document.

term	$tf_{t,d}$	idf_t	$tf - idf_{t,d}$
`calculus` in d_1	0.091	1.386	0.126
`linear` in d_2	0.2	1.386	0.277
`literature` in d_3	0.143	1.386	0.198

Model Setup Once each course description is represented as a TF-IDF vector, these numerical features serve as the input to the logistic regression classifier. *Logistic Regression* models the relationship between the TF-IDF features and the probability that a description belongs to the MATH category.

Let $X = (X_1, X_2, \ldots, X_p)$ denote the TF-IDF weights for all vocabulary terms in a single course description, where each X_i represents the TF-IDF value for term i. The logistic regression model will predict the probability that this description belongs to Math as

$$\text{logit}(p(X)) = \ln\left(\frac{p(X)}{1 - p(X)}\right)$$

$$= \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$

Weights into Probability Once each description is converted into its TF-IDF feature vector, the logistic regression model combines these weighted tokens into a singular numerical score, which determines the probability that the description belongs to the MATH category. Let $X = (X_1, X_2, \ldots, X_p)$ denote the TF–IDF values for all vocabulary terms in one description. Logistic regression uses these values and the fitted coefficients to compute the log-odds of a MATH classification:

$$\text{logit}(p(X)) = \beta_0 + \sum_{i=1}^{p} \beta_i X_i$$

Positive coefficients increases the log-odds of predicting Math when their corresponding terms appear with nonzero TF-IDF weights, while negative coefficients push the prediction toward ENLT.

Unlike the earlier example, which focused on how individual coefficients relate to specific terms, this example shows how the model combines all weighted features to produce an overall probability. To illustrate the application of this, consider the fitted model applied to the following description (not included in the training):

> Topics include sampling distributions, estimation, theory of estimators, test of hypotheses, analysis of variance, regression and correlation analysis, time series, experimental design, modeling and decision criteria. The use of statistical analysis in decision problems is stressed. Prerequisite: MATH 345 or equivalent.

For this description, the tokens with the largest positive contributions to the log-odds were:

$$\texttt{math}\ (0.078), \quad \texttt{modeling}\ (0.034), \quad \texttt{topics}\ (0.033), \quad \texttt{prerequisite}\ (0.032).$$

Terms with negative contributions included:

$$\texttt{time}\ (-0.007).$$

When the TF-IDF values for these terms are multiplied by their corresponding coefficients and summed, the resulting log-odds correspond to a predicted probability of

$$p(X) = 0.585.$$

Since this value is above 0.5, the model classifies the description as MATH.

It is important to note that the log-odds transformation used here is a component of logistic regression itself and is separate from the logarithm used in the IDF formula. The IDF logarithm is used only to quantify the rarity of individual terms across the corpus, while the logistic logit is used to convert the combined TF–IDF features score into a valid probability. A fuller analysis of the fitted coefficients appears in Section 4.1

How the Model Learns The model's learning process involves estimating the coefficients β_i using *maximum likelihood estimation*. At a high level, this technique selects the values of β that

make the observed labels (MATH or ENLT) most probable under the logistic regression model. Taking the logarithm of likelihood converts products over many observations into a sum, which is easier to optimize, and leads to the log-odds form of the logistic regression equation. Although it is a bit more out of the scope of the project, it is still important to note that maximum likelihood finds the set of β values that maximize the probability of concluding the correct labels in the training data. At the start, the model assumes no prior information, meaning each description is equally likely to be Math to English, which follows along with the first assumption of an initial probability of $P_t = 0.5$. The roles of TF-IDF and logistic regression can be viewed in three steps:

TF-IDF gives every word a fixed important score based on how often it appears in the corpus.

Logistic Regression then learns separate coefficients (the β's) that show how strongly each word pushes the prediction toward Math or English.

The final "influence" of a word is TF-IDF $\times\beta$

So, TF-IDF and logistic regression play complementary roles, but they operate at different stages in the process. Logistic regression just learns an additional set of coefficients β_i that determine how strongly that word should influence the final prediction, which stems from the initial measure that TF-IDF gives on how informative a word is across a corpus.

3.3 Implementation

The full modeling pipeline was implemented in Python using the `pandas` and `scikit-learn` libraries. The dataset was first loaded from a CSV file, and all course description were standardized in code by replacing missing values, converting entries to strings, and stripping extra whitespace. A TF-IDF vectorizer was then initialized with English stop-word removal and a custom token pattern that preserves hyphenated words. By applying `vec.fit_transform`, it learned the vocabulary and converted each description into a sparse TF-IDF vector.

Next, the labels (`MATH` and `ENLT`) were mapped to numeric values and a logistic regression model was trained on the TF-IDF matrix using a higher `max_iter` value to help ensure combination with many features. After fitting, the mode's coefficients were paired with the vectorizer's vocabulary to identify the most influential terms for predicting each class. Throughout the code, there were `print blocks` to give examples of predictions, extracted class probabilities, and the contributions for a few different tokens for a new course description.

All code used for loading the data, vectorizing the descriptions, training the model, and examining the coefficients is in the Appendix.

4 Results

4.1 Vocab and Feature Analysis

After fitting the logistic regression model to the TF-IDF vectors, each term in the vocabulary gets a coefficient β_i that reflects how strongly the presence of that word contributes to predicting the Math (1) versus English Literature (0) category. Sorting these coefficients allows us to identify the most informative terms for each class. The top ten words with the largest positive coefficients (most indicative of MATH) and top ten words with the most negative coefficients (indicative of ENLT) are shown below. These values come directly from the fitted model.

Top MATH terms:

equations (0.194)

rational (0.202)

prerequisite (0.210)

topics (0.215)

emphasis (0.218)

geometry (0.219)

modeling (0.219)

mathematical (0.274)

linear (0.288)

math (0.544)

Top ENLT terms:

literature (-0.325)

literary (-0.324)

writing (-0.223)

women (-0.218)

readings (-0.205)

contemporary (-0.200)

animals (-0.186)

health (-0.186)

1960s (-0.181)

writers (-0.181)

These results align well with common expectations from the course descriptions. The Math terms tend to refer to mathematical and quantitative topics, such as *geometry*, *linear*, or *modeling*, while the English Literature terms tend to reference concepts, such as *literature*, *writers*, or *contemporary*.

4.2 Classification Performance

To evaluate the effectiveness of the model, the trained logistic regression classifier was applied to all twenty course descriptions in the datatset. Each document was converted to TF-IDF features using the learned vocabulary, and the model predicted both a class label and a probability for each description. Comparing these predictions with the true labels shows that the classifier correctly identified every Math and English Literature course:

$$\text{Correct predictions} = 20, \qquad \text{Incorrect predictions} = 0.$$

Thus, the overall accuracy on the dataset is

$$\text{accuracy} = 1.00.$$

The confusion matrix will look like

	Predicted ENLT	Predicted MATH
Actual ENLT	10	0
Actual MATH	0	10

Although the dataset is small, this outcome indicates that the TF-IDF features show clear distinctions between the two categories, and logistic regression is able to use those features.

4.3 Interpretation

To show how we can examine how individual words within a course description influence its predicted label, consider the model's output for a new description not included in the training data. After transforming the description into TF–IDF features, the model predicted the label `MATH` with an estimated probability of approximately 0.585.

> Topics include sampling distributions, estimation, theory of estimators, test of hypotheses, analysis of variance, regression and correlation analysis, time series, experimental design, modeling and decision criteria. The use of statistical analysis in decision problems is stressed. Prerequisite: MATH 345 or equivalent.

```
Predicted label: MATH
Class probabilities (ENLT, MATH): [0.41496103 0.58503897]
Predicted probability of MATH: 0.5850389723167728
```

For this new description, only a subset of the vocabulary appears with nonzero TF-IDF weights. For each token, after multiplying the TF-IDF value by the coefficient β_i to get the weight, sorting the weight contributions reveals the terms that affected the decision the most.

Top positive (push toward MATH):	**Top negative (push toward ENLT):**
math (0.078)	time (-0.007)
modeling (0.034)	design (0.014)
topics (0.033)	equivalent (0.014)
prerequisite (0.032)	estimation (0.014)
theory (0.029)	experimental (0.014)

These contribution values reflect how TF-IDF and logistic regression work together and the final predicted probability of 0.585 reflects this.

5 Discussion

5.1 Strengths

A key strength of using TF-IDF with logistic regression is its interpretability. Each word receives a clear numerical weight, and the logistic model assigns coefficients that show exactly how those weights influence the final probability. This makes it easy to identify which terms matter most and why a prediction leans towards Math or English Literature.

Another strength is that TF-IDF captures term important in a simple, transparent way. We now know that frequent but uninformative words receive lower weights, while distinctive words receive higher ones. This creates a straight forward, easy to understand, and explainable decision process.

Finally, the overall workflow (preprocessing, vectorization, modeling, etc) matches the same foundation of more advanced Natural Language Processing systems, which provides a clear conceptual basis to understanding larger models.

5.2 Limitations

One main limitation is the bag of words structure, which ignores word order, context, and meaning of words. As a result, two phrases with very different meanings can look similar if they share some of the same words.

Also, TF-IDF treat each term independently, without modeling possible concepts, meanings, or interactions between words or sequences. This also links off the bag of words structure limitation. Moving on to further and more complex language models, such as RNNs and Transformers, this is where we are able to capture relationships across time, position, or context.

6 Conclusion

This project began with a broad interest in how machines understand language. During the first talk, the goal was to learn how machines understand and generate language, following the same road map that leads to Large Language Models (LLMs). The focus was defining key ideas in Machine Learning and Natural Language Processing, and understanding models such as Logistic Regression, Recurrent Neural Networks (RNNs), and Transformers. The goal was to explore how these transform from simple rule based systems into the complex systems used today.

In the second talk, the focus shifted to applying these ideas to a real dataset. The TensorFlow IMDB sentiment classification tutorial, which helped to understand the workflow of these models. This introduced loading and splitting data, vectorizing text, training a model, and evaluating performance.

Through that initial research and faculty feedback, the realization of how complex and advanced these models are came to light. It was decided that it was more important to build a stronger understanding of the mathematical foundations that appear in simple, interpretable models such as logistic regression. This strong base will allow for better learning and understanding when it comes to the more complex models.

The scope was finally narrowed to TF-IDF. This final implementation combined with Logistic Regression reflects on that decision to complicity the scope of this project. It still had the same principles used by RNNs and Transformers, but in a way that allows every step to be explained and easily understood. Rather than going for the highest complexity models, it was learned how important it is to have a clear, strong, foundational mathematical understanding. This shift in scope also highlights one of the initial goals: not just knowing that a model works, but why it works.

In that sense TF-IDF and logistic regression serve as a concrete step towards understanding larger language models. Both rely on representing text numerically, weight informative terms, and

learning parameters from data. The work in this project shows how these ideas can be carried out in full detail for a small, interpretable model, which sets the stage for later study of RNNs, Transformers, and the LLMs builts from all of them.

Appendices

A Conceptual Walk-Through: From Raw Text to Prediction

This appendix collects a concrete, end-to-end example showing how a course description moves from raw text to a numeric representation and then into the logistic regression model. The example uses one MATH and one ENLT course from `smc_courses.csv` dataset and focuses on the key quantities and computations.

A.1 Start with Raw Text

We consider two sample entries from the dataset:

label	course id	title	description (truncated)
MATH	MATH 103	Precalculus	This course studies polynomial, rational, exponential, logarithmic, and trigonometric functions from the symbolic, numeric, and graphical perspectives. The emphasis on these concepts will provide solid preparation for a college-level calculus course...
ENLT	ENLT 216	Literature of Social Justice	What role do literary works play in advocating for a more just society? Can literature provide a platform for social exploration? A space for critique and disruption? This course uses these guiding questions to examine works of poetry, fiction, and nonfiction...

Table 1: Sample entries from the *smc-courses* dataset. Descriptions truncated for display.

To understand what happens in the next step, think of the phrase

"Introduction to Statistics"

which, becomes the list

[`introduction`, `to`, `statistics`]

Each description is treated as the *bag of words*, where order is ignored and frequency matters. Furthermore, it is important that the concept of removing *stop words* is implemented. Thus, the vector becomes

$$[\texttt{introduction}, \texttt{statistics}]$$

After lowercasing, removing punctuation, and tokenizing, a portion of the MATH 103 description becomes a list of tokens, such as

$$[\texttt{course}, \texttt{studies}, \texttt{polynomial}, \texttt{rational}, \texttt{exponential}, \texttt{logarithmic}$$

$$\texttt{trigonometric}, \texttt{functions}, \texttt{symbolic}, \texttt{numeric}, \texttt{graphical}, \texttt{perspectives}, \dots].$$

Common stop words are removed, and the remaining tokens are the starting point for the TF-IDF weights.

A.2 Term Frequency (TF)

For each term t in a description d, the term frequency is

$$tf_{t,d} = \frac{\text{number of times } t \text{ appears in } d}{\text{total number of terms in } d}$$

For example, if the word `functions` appears once (giving it a small term frequency) in a 50-word description for MATH 103, then

$$tf_{\text{functions, MATH103}} = \frac{1}{50} = 0.02$$

This means that the word *functions* makes up 2% of the total words in this description. Again, a larger term frequency indicates the word is more prominent within that give description. Common words like *as* or *the* have a higher frequency, but eventually will be down weighted.

A.3 Inverse Document Frequency (IDF)

With N denoting the total number of course descriptions and df_t the number of descriptions containing term t, the IDF is

$$idf_t = \log\left(\frac{N}{df_t}\right)$$

In this project, $N = 20$. If `functions` appears in $df_t = 4$ descriptions, then

$$idf_{\text{functions}} = \log\left(\frac{20}{4}\right)$$

$$= \log(5) \approx 0.6999.$$

If `functions` appears in 12 descriptions, then

$$idf_{\text{course}} = \log\left(\frac{20}{12}\right)$$

$$= \log(1.67) \approx 0.222.$$

A.4 Combining into TF-IDF Weights

The TF-IDF weight for term t in document d is

$$tfidf_{t,d} = tf_{t,d} \times idf_t$$

Continuing the example:

For the word `functions` with $tf = 2/40 = 0.04$ and $idf = \log(20/4) \approx 0.699$,

$$tf_{\texttt{functions},\,\text{MATH103}} = 0.04 \times 0.699 \approx 0.028.$$

For the word `course` with $tf = 3/50 = 0.06$ and $idf = \log(20/12) \approx 0.222$,:

$$tf_{\text{course},\,\text{MATH103}} == 0.06 \times 0.222 \approx 0.013.$$

Thus, `functions` receives a larger TF-IDF weight than `course` in the same description.

A course description d is then represented by the TF-IDF vector

$$X_d = [\, tfidf_{\text{functions},d},\, tfidf_{\text{course},d},\, tfidf_{\text{calculus},d},\, \ldots]$$

A.5 Logistic Regression Model Setup

Once each description is represented by a TF-IDF vector, logisitic regression models the probability that the description belongs ot the MATH category as

$$\log\left(\frac{p(X_d)}{1-p(X_d)}\right)$$

$$= \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$

so that

$$p(X_d) = \frac{e^{\beta_0+\beta_1 X_1+\beta_2 X_2+\cdots+\beta_p X_p}}{1+e^{\beta_0+\beta_1 X_1+\beta_2 X_2+\cdots+\beta_p X_p}}$$

Each coefficient β_i corresponds to a specific term in the vocabulary. Positive coefficients increase the odds of predicting MATH as the TF–IDF weight of that term increases, and negative coefficients increase the odds of predicting ENLT.

Suppose MATH 103 is represented by two TF-IDF values

$$X_{\texttt{functions}} = 0.028, \quad X_{\texttt{course}} = 0.013,$$

with coefficients

$$\beta_{\text{functions}} = 1.2 \quad \text{and} \quad \beta_{\text{course}} = -0.8.$$

The weighted sum for the MATH 103 description is

$$\text{weighted sum} = \beta_0 + \beta_{\text{functions}} X_{\text{functions}} + \beta_{\text{course}} X_{\text{course}}$$

$$\text{weighted sum} = 0 + (1.2)(0.028) + (-0.8)(0.013)$$

$$= 0.0336 - 0.0104$$

$$= 0.0232$$

and the corresponding probability is

$$p(X_d) = \frac{1}{1 + e^{-\text{weighted sum}}}$$
$$= \frac{1}{1 + e^{-0.0232}}$$
$$\approx 0.506$$

A.6 Interpreting Coefficients through Odds Ratio

Exponentiating a coefficient returns its odds ratio. Applying the exponential to a coefficient transforms it from the log odds scale, back into just the odds scale, which helps to make it more intuitive. For instance, if $\beta_{\text{functions}} = 1.05$, then

$$e^{1.05} \approx 2.86,$$

so a one-unit imcrease in the TF-IDF value of `functions` multiplies the odds ratio of the course being classified as MATH by about 2.9.

If $\beta_{\text{course}} = -0.35$, then

$$e^{-0.35} \approx 0.70,$$

meaning that higher TF-IDF values for `course` slightly decreases the odds that a description is MATH.

B Implemented Model Using Python

This appendix summarizes the Python Implementation that follows the conceptual walk-through. The code uses `pandas`, `NumPy`, and `scikit-learn` to carry out the steps:

1. Load the `smc_courses.csv` file and clean the `description` field.

2. Vectorize all descriptions with a `TfidfVectorizer` (English stop words, custom token pattern for hyphenated words, etc).

3. Encode labels as `ENLT` = 0 and `MATH` = 1.

4. Fit a `LogisticRegression` model on the TF–IDF matrix.

5. Extract the learned coefficients and list the top positive and negative terms.

6. Apply the model to a new description, showing its predicted label and class probabilities.

The following appendices contain selected code chunks used to implement these steps.

C Setup and Cleaning Code

This appendix shows the core code that is responsible for cleaning the descriptions and constructing the TF-IDF matrix.

We construct a *TfidfVectorizer* with three key parameters:

1. Remove English stopwords (i.e. 'the', 'a', 'or', etc.)

2. Keep hyphenated words together

3. Use only single words

```
# Replace missing descriptions, ensure string type, strip whitespace
df['description'] = df['description'].fillna('').astype(str).str.strip()
```

We then work to get the sparse matrix, by calling `fit_transform`, where the entries are the TF-IDF weights.

```
    vec = TfidfVectorizer(
    stop_words='english',   # setting for all (the, a, in, on, etc.) words
    token_pattern=r"(?u)\b\w+(?:-\w+)*\b",  # keep hyphenated words together
    ngram_range=(1,1), #single words
    # max_features=30 # this caps the number of words to use as features in model
)
```

```
    from sklearn.feature_extraction.text import TfidfVectorizer

    vectorizer = TfidfVectorizer(stop_words='english')
    X = vectorizer.fit_transform(corpus)
    print(X.shape)
```

The TF-IDF matrix X has a shape $(20, 451)$ with 605 stored (nonzero) elements. We save this as our alphabet and to use for analysis. The sparsity of the matrix is therefore

$$\begin{aligned}\text{sparsity} &= \frac{\text{stored elements}}{\text{number of descriptions} \times \text{number of unique tokens}} \\ &= \frac{605}{20 \times 451} \\ &= 0.067\end{aligned}$$

The learned vocabulary consists of 451 unique tokens. A sample of feature names is:

$$[\texttt{"101"}, \texttt{"103"}, \texttt{"104"}, \texttt{"105"}, \texttt{"113"}, \texttt{"121"}, \texttt{"1960s"}, \texttt{"19th"}, \texttt{"2004"}, \texttt{"2023"},$$

$$\texttt{"20th"}, \texttt{"3"}, \texttt{"4"}, \texttt{"absolute"}, \texttt{"adaptations"}, \ldots].$$

And the head of the matrix looks like

	label	101	103	104	105	113	121	1960s	19th	2004	...	voices	w	watch	ways	women	works	writers	writing	writings	year
0	ENLT	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.000000	...	0.000000	0.234049	0.000000	0.000000	0.000000	0.0	0.0	0.211193	0.000000	0.0
1	ENLT	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.089385	0.089385	...	0.000000	0.000000	0.089385	0.000000	0.000000	0.0	0.0	0.000000	0.000000	0.0
2	ENLT	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.000000	...	0.186929	0.164314	0.000000	0.000000	0.164314	0.0	0.0	0.296535	0.000000	0.0
3	ENLT	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0	0.000000	0.000000	0.0
4	ENLT	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.000000	...	0.000000	0.000000	0.000000	0.248461	0.000000	0.0	0.0	0.000000	0.141329	0.0

5 rows × 452 columns

D Model Training Code (Logistic Regression)

This appendix shows the main training and analysis code for the logistic regression model. We want to encode labels to fit a logistic regression classifier on X. To do this, we first need to convert our labels into numbers that the model can understand. Logistic regression can only work with numerical inputs, so we assign:

$$\text{ENLT} = 0 \text{ and } \text{MATH} = 1$$

```
from sklearn.linear_model import LogisticRegression

# create numeric labels
```

```
# store in NumPy array as integers
label2id = {"ENLT":0, "MATH":1}
y = df["label"].map(label2id).astype(int).to_numpy()

# Initialize and fit logistic regression model
model = LogisticRegression(max_iter=2000)
model.fit(X, y)
```

Once the labels are set, we can train our logistic regression model using TF-IDF matrix X and the labels y. The model will learn a coefficient for each word in the vocabulary, which represents how much that word influences the prediction.

```
    # Feature names from the vectorizer
    # Examine features and return a list of all the words in vocabulary
    feature_names = vec.get_feature_names_out()

    # Store the coefficients fit to each feature, i.e., every word in vocabulary
    # .ravel() flattens the result into a 1D array
    coefs = model.coef_.ravel()

    # We want to find the most informative coefficients for classifying Math
        descriptions, so look at the 10 largest coefficients
    top_math = coefs.argsort()[-10:]

    # For English courses, we want the smallest coefficients
    top_enlt = coefs.argsort()[:10]
```

These lists produce the "Top MATH terms" and "Top ENLT terms" reported in the Results section. It is important to note that the default number of iterations (100) is too low for any text data with many features, so we want to increase it to make sure the model can successfully find a solution.

Prediction for a New Description

```
    new_desc = """Topics include sampling distributions, estimation, theory of
        estimators, test of hypotheses, analysis of variance, regression and
        correlation analysis, time series, experimental design, modeling and
        decision criteria. The use of statistical analysis in decision problems is
        stressed. Prerequisite: MATH 345 or equivalent."""

    X_new = vec.transform([new_desc])
```

```
    pred_id = model.predict(X_new)[0]
    proba = model.predict_proba(X_new)[0]  # array([P(class 0), P(class 1)])

    id2label = {0: "ENLT", 1: "MATH"}
    pred_label = id2label[pred_id]

    print("Predicted label:", pred_label)
    print("Class probabilities (ENLT, MATH):", proba)
    print("Predicted probability of MATH:", proba[1])
```

In this example, the model predicts label `MATH` with probability ≈ 0.585 for the new course description. The same process is used to analyze the contribution of individual words to this predictions in the Results section.

References

[1] TensorFlow Core, "Basic Text Classification," *TensorFlow*, n.d., https://www.tensorflow.org/tutorials/keras/text_classification.

[2] François Chollet, *Deep Learning with Python (Third Edition)*, Manning Publications, 2023.

[3] IBM, "What are Large Language Models (LLMs)?", *IBM Think Blog*, 2023, https://www.ibm.com/think/topics/large-language-models.

[4] IBM, "Recurrent Neural Networks (RNNs)," *IBM Think Blog*, 2023, https://www.ibm.com/think/topics/recurrent-neural-networks.

[5] IBM, "What is Logistic Regression?", *IBM Think Blog*, 2023, https://www.ibm.com/think/topics/logistic-regression.

[6] GeeksforGeeks, "Getting Started with Transformers," *GeeksforGeeks Machine Learning*, 2023, https://www.geeksforgeeks.org/machine-learning/getting-started-with-transformers/.

[7] Amazon Web Services, "What are Transformers in Artificial Intelligence?", *AWS*, 2023, https://aws.amazon.com/what-is/transformers-in-artificial-intelligence/.

[8] Two Minute Papers, "Transformers Explained: What They Are and How They Work," *YouTube*, 2023, https://www.youtube.com/watch?v=zxQyTK8quyY.

[9] IBM, "Text Classification," *IBM Think Blog*, 2023, https://www.ibm.com/think/topics/text-classification.

[10] Stack Exchange, "What Exactly is Multi-Hot Encoding and How is it Different from One-Hot?," *Cross Validated (Stack Exchange)*, 2022, https://stats.stackexchange.com/questions/467633/what-exactly-is-multi-hot-encoding-and-how-is-it-different-from-one-hot.

[11] Stack Overflow, "Import Data into Google Colaboratory," *Stack Overflow Discussion*, 2017, https://stackoverflow.com/questions/46986398/import-data-into-google-colaboratory.

[12] Saturn Cloud, "How to Import Files from Google Drive to Colab," *Saturn Cloud Blog*, 2023, https://saturncloud.io/blog/how-to-import-files-from-google-drive-to-colab/.

[13] V7 Labs, "Train, Validation, and Test Sets in Machine Learning," *V7 Labs Blog*, 2022, https://www.v7labs.com/blog/train-validation-test-set.

[14] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze, "Term Frequency–Inverse Document Frequency (TF–IDF)," *Introduction to Information Retrieval*, Cambridge University Press, 2008, Section 6.2, http://nlp.stanford.edu/IR-book/pdf/06vect.pdf.

[15] Stanford NLP Group, *The Stanford Information Retrieval Book*, Stanford University, https://www-nlp.stanford.edu/IR-book/.